

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 01 -

**- SAÚDE DA APLICAÇÃO E
GERENCIAMENTO DE RECURSOS**

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	18
O QUE VOCÊ VIU NESTA AULA?	18
REFERÊNCIAS.....	23

O QUE VEM POR AÍ?

Bem-vindos e bem-vindas! Nesta seção, vamos explorar como o Kubernetes garante a confiabilidade de suas aplicações através de mecanismos vitais de monitoramento. Investigaremos o papel das "probes" (sondas), entendendo como a *Liveness Probe* detecta travamentos, a *Readiness Probe* gerencia o tráfego para contêineres prontos e a *Startup Probe* protege aplicações de inicialização lenta. Além disso, mergulharemos no gerenciamento de recursos, desvendando como *Requests* (requisições) e *Limits* (limites) de CPU e memória são cruciais para a estabilidade. Através de exemplos práticos, veremos como a configuração correta desses elementos pode prevenir falhas e otimizar a performance no dia a dia.

HANDS ON

Nesta seção, vamos construir e implantar um microserviço em Rust do zero, configurando na prática as probes de saúde e os limites de recursos que discutimos.

Primeiro, criaremos uma API web simples em Rust com dois endpoints essenciais: `/healthz`, que simplesmente confirma que o processo está rodando, e `/ready`, que simulará uma verificação de dependências. Em seguida, vamos compilar o código e empacotá-lo em uma imagem Docker leve e otimizada, pronta para produção.

O passo central será a criação do manifesto de Deployment no Kubernetes. Nele, você irá configurar detalhadamente cada probe: a *startup probe* para dar tempo à aplicação iniciar, a *liveness* para reiniciar o contêiner se ele travar, e a *readiness* para tirá-lo do serviço se uma dependência falhar. Também definiremos `requests` e `limits` de CPU e memória para garantir um uso de recursos previsível.

Com o manifesto pronto, você aplicará tudo no cluster com `kubectl apply` e aprenderá a usar comandos como `kubectl describe pod` para inspecionar os eventos e confirmar que as probes estão funcionando. Para finalizar, vamos simular falhas intencionalmente para observar em tempo real o Kubernetes agindo, provando o poder e a resiliência que essas configurações oferecem.

Vale destacar como a escolha de Rust se alinha ao gerenciamento de recursos. Rust gera binários compilados extremamente otimizados – o consumo de CPU tende a ser previsível, sem picos de garbage collection.

Além disso, a segurança de memória do Rust previne acessos inválidos e muitos vazamentos comuns, reduzindo a chance de crashes ou comportamentos indefinidos em produção. Assim, conseguiremos rodar nosso serviço confortavelmente dentro de 100–200m de CPU e <128Mi RAM, mesmo sob carga, onde outra linguagem talvez exigisse mais recursos para a mesma tarefa.

Grandes empresas já adotam Rust em componentes de infraestrutura crítica exatamente por essa eficiência; por exemplo, o Linkerd2, popular service mesh,

reescreveu seu proxy em Rust para reduzir drasticamente uso de memória e latência, permitindo maior densidade de pods por nó.

Portanto, usar Rust nos permite definir requests/limits mais baixos, aumentando a economia de recursos no cluster sem sacrificar performance – um ganha-ganha para Dev e Ops, mesmo se comparado a linguagens mais populares como Go, Java e C#.

EMANIP

SAIBA MAIS

Nesta seção, mergulharemos em conceitos avançados de ciência da computação e engenharia de software, conectando-os ao contexto de containers e Kubernetes. Veremos modelos matemáticos de disponibilidade, detalhes de gerenciamento de memória/CPU (como *garbage collection*, preempção, escalonamento justo) e até um pouco de estatística aplicada a sistemas distribuídos – tudo isso de forma descontraída, mas **densa** em conteúdo. A ideia é reforçar os princípios teóricos que fundamentam as práticas de *probes* e gerenciamento de recursos que discutimos, enriquecendo sua compreensão sobre *porque* essas ferramentas funcionam e como otimizar seu uso.

Modelos de disponibilidade e falhas (Markov Chains e probabilidade)

Quando consideramos a **confiabilidade** de aplicações em um cluster, podemos modelar seus estados de funcionamento usando conceitos probabilísticos. Um modelo clássico é a **Cadeia de Markov** de dois estados: *Up* (funcionando) e *Down* (falha). Suponha que um determinado pod tem taxa de falha λ (lambda) e taxa de recuperação μ (mu) – ou seja, em média λ falhas por unidade de tempo e, quando falha, volta ao ar após um tempo médio $1/\mu$, seguindo uma função densidade de probabilidade Exponencial ($P(x) \sim \text{Exp}(\lambda)$, onde x é a probabilidade de ocorrer um evento de falha). Supondo que as falhas são aleatórias (ou seja, elas podem ocorrer por um número de eventos não discriminatórios e independentes) e independentes do passado (propriedade Markoviana), podemos calcular a **disponibilidade de estado estacionário** desse serviço como $\mu / (\lambda + \mu)$.

Essa fórmula é equivalente a $MTBF / (MTBF + MTTR)$ (Mean Time Between Failures dividido pela soma com Mean Time To Repair). Isso por que $MTBF = \frac{1}{\lambda}$ e $MTTR = \frac{1}{\mu}$, logo

$$\frac{MTBF}{(MTBF + MTTR)} = \frac{\frac{1}{\lambda}}{\frac{1}{\lambda} + \frac{1}{\mu}} = \frac{\frac{1}{\lambda}}{\frac{\lambda + \mu}{\lambda\mu}} = \frac{\frac{\lambda\mu}{\lambda}}{\frac{\lambda + \mu}{1}} = \frac{\mu}{\lambda + \mu}$$

Por exemplo, se em média um pod roda 100 horas antes de falhar (MTBF) e demora 1 hora para ser recuperado (MTTR), sua disponibilidade teórica = $100/(100+1) \approx 99\%$. Esse tipo de cálculo ajuda a quantificar metas de SLA e a entender o impacto

de melhorar tempos de reparo – algo que o Kubernetes faz de forma automatizada ao reiniciar pods em segundos quando detecta falhas (via liveness probe), diminuindo o MTTR e assim elevando a disponibilidade total.

Em arquiteturas de microsserviços, também lidamos com **probabilidade condicional e composição de disponibilidades**. Componentes em série e paralelos têm comportamentos diferentes: se seu sistema precisa de 3 serviços em sequência para atender uma requisição, ele só estará disponível se **todos os 3** estiverem disponíveis simultaneamente (modelo em série). A disponibilidade combinada é o produto das disponibilidades individuais. Por exemplo, três serviços com 99% de uptime cada resultam em $\sim 0,99^3 = 97\%$ de uptime fim-a-fim – piora conforme encadeamos dependências. Já se temos componentes redundantes em paralelo (por exemplo, dois pods servindo a mesma função atrás de um *Service*), a disponibilidade sobe, pois basta um dos dois estar ativo para o serviço funcionar.

A fórmula de disponibilidade em paralelo (para falhas independentes) é $A_N = 1 - \prod_{i=1}^N 1 - A_i$. Assim, dois pods com 99% de disponibilidade cada alcançam **99.99%** de disponibilidade conjunta (0,01% de downtime apenas), e 10 pods com 50% cada (!) poderiam teoricamente fornecer 99,9% – isto ilustra por que replicar pods (escalonamento horizontal) melhora a resiliência. Claro, supõe independência de falhas – na prática, pode haver causas comuns que derrubem vários pods (ex: bug na imagem ou nó em pane).

Ainda assim, Kubernetes tira proveito desse paralelismo ao possibilitar múltiplas réplicas e ao usar *avaliações de disponibilidade* para remover rapidamente da rotação qualquer réplica com problemas. O serviço do Kubernetes funciona como um *or* lógico: encaminha tráfego para **pelo menos um** pod *ready* – isso é justamente disponibilidade em paralelo, onde o sistema se mantém atendendo se pelo menos um membro do conjunto está OK.

Podemos até modelar *degradação de desempenho* como estados adicionais em uma cadeia de Markov (por exemplo: estado *Healthy*, *Degraded* e *Down*). Ferramentas como o **Prometheus** e **Grafana** permitem coletar métricas de erro e construir modelos simples: por exemplo, a cada tentativa de requisição temos uma probabilidade $P(x) \sim \text{Exp}(\lambda)$ de falha transitória; se falhar, usamos mecanismos de repetição exponencial (*exponential backoff*) – podemos calcular a probabilidade

condicional de sucesso após n tentativas ($P(x_n|x_{n-1})$) de recuperação, que é exponencialmente menor do que a ocorrência da primeira falha (se o sistema for estocástico).

Quando existem razões determinísticas para a falha, ela é identificada e corrigida de acordo com o tempo de desenvolvimento e correção de falhas, governado a nível de gerenciamento do projeto e, portanto, é bem-definido e delimitado para a redução do *downtime*. Por isso, o controle de ambiente é fundamental para a garantia e estabilidade de um sistema Kubernetes: o serviço K8s é eficiente no controle de falhas aleatórias, mas não governa falhas determinadas por operações humanas (como desenvolvimento e planejamento da aplicação).

Esses conceitos de estatística nos lembram que sistemas distribuídos não são apenas determinísticos – há sempre uma distribuição de tempos de resposta, probabilidades de falha e recuperação. Administrar isso requer pensar em termos de SLOs (Service Level Objectives) probabilísticos, e usar readiness/liveness probes adequadamente ajuda a manter essas probabilidades sob controle (por ex., removendo temporariamente instâncias com erro, aumentamos a probabilidade de atender requisições com sucesso via outras instâncias redundantes).

Em termos de verificação de estado, podemos dizer que as probes implementam um “algoritmo” simples: testar periodicamente e inferir o estado do app com base no último resultado (sucesso/falha) – é quase como um processo estocástico de dois estados sendo observado. A diferença é que as probes não preveem o futuro, mas atuam reativamente: se falhar, marca *down*, se passar, marca *up*:


```
state := UNKNOWN
cf := 0; cs := 0
sleep(initialDelay)

every period P:
    result := check_with_timeout(timeout)
    if result == SUCCESS:
        cs := cs + 1; cf := 0
        if state != UP and cs >= s:
            state := UP
            on_up()
    else: // FAIL or TIMEOUT
        cf := cf + 1; cs := 0
        if state != DOWN and cf >= f:
            state := DOWN
            on_down()
```

Podemos relacionar isso com conceitos de **detecção de falhas** em sistemas distribuídos: existem algoritmos e modelos (como *heartbeat detectors* e *phi-accrual algorithms*) que interpretam sequências de *heartbetas* para estimar a probabilidade de um nó ter falhado. O kubelet no fundo está fazendo um tipo de checagem de *heartbeat* check com as probes HTTP/exec, seguindo um modelo bem simples (falha × vezes consecutivas => declarar falha). Apesar da simplicidade, esse esquema é eficaz para a maioria dos casos, e alinhado com o modelo de falha binária (vivo/morto) que o orquestrador assume.

Gerenciamento de memória, Garbage Collection e preempção de CPU

No ambiente de containers, gerenciamento eficiente de memória e CPU é crucial. Aqui entram conceitos de **ciência da computação** como *garbage collection* (GC), alocação de memória, e algoritmos de escalonamento e priorização de CPU. Como vimos, Rust evita um *garbage collection* pausando a aplicação – memória é liberada determinística e rapidamente quando variáveis saem de escopo de

processamento, minimizando a quantidade de memória necessária para a execução de um programa.

Em linguagens com GC (Java, Go e C#), o *runtime* gerencia a memória, mas pode introduzir pausas (as famosas *STW stops*) durante a coleta. Essas pausas podem impactar a *liveness probe* se coincidirem com a janela de checagem: por exemplo, imagine que o GC entrou em ação e travou os threads por 300ms, e justamente nesse momento o kubelet fez um HTTP GET na avaliação de disponibilidade com timeout de 200ms – do ponto de vista do orquestrador, o app não respondeu a tempo e aquela tentativa conta como falha. Se isso ocorrer repetidamente, pode levar a reinício do pod indevidamente.

Ainda que improvável em baixo volume de requisições, essa indisponibilidade possível se torna consistentemente mais provável em alta escala de requisição. Por isso, ao configurar timeouts das probes devemos considerar o perfil da linguagem e *runtime*, considerando que a aplicação tenha escala exponencial de utilização, e o tratamento deste cenário precisa ser considerado ao definir o perfil de probes do Kubernetes.

Em Java, técnicas como *tuning* do GC ou uso de coletores de baixa latência (ZGC, Shenandoah) podem mitigar pausas longas. Já em Go, o GC é incremental, mas ainda pode causar pequenos spikes de CPU que, se a aplicação estiver no limite da CPU, podem disparar *throttling* e aumentar latência. Um gráfico comum de se analisar é **consumo de CPU vs tempo de resposta**: conforme CPU se aproxima de 100% de utilização, a latência média e – mais importante – a latência de cauda (p95, p99) tende a explodir.

Em Kubernetes, se definimos *limites* muito apertados, podemos cair na situação de *throttling de CPU*: o kernel atrasa a execução do processo para mantê-lo dentro do limite, introduzindo latências artificiais. O blog post do Buffer mostrou isso claramente: mesmo com baixo uso médio de CPU, o simples fato de haver um limite fez o *throttling* do kernel acontecer milhares de vezes, adicionando latência à aplicação e à sua disponibilidade, tanto por questão de volume de requisição quanto por perfil de linguagem adotada.

Neste caso, o conselho dos engenheiros da Buffer foi remover os limites de CPU em serviços sensíveis à latência e confiar apenas nas requisições e no

monitoramento, mantendo os limites altos para serviços essenciais de checagem sistêmica. Esse é um ótimo exemplo de como princípios de escalonamento de CPU (justo, mas com quotas) podem impactar estatísticas de desempenho, disponibilidade e escalabilidade de uma aplicação em Kubernetes.

No tópico de **escalonamento justo**, vale lembrar como o kernel Linux (subjacente aos sistemas Kubernetes) gerencia CPU entre processos: ele usa o algoritmo **CFS (Completely Fair Scheduler)**, que basicamente tenta dividir o tempo de CPU de forma proporcional aos pesos (shares) de cada processo.

Em Kubernetes, quando configuramos `resources.requests.cpu`, o kubelet traduz isso em valores de `cpu.shares` no cgroup do contêiner. Um contêiner com maior carga de requisições disponível tem mais peso e ganha mais fatias de CPU em situação de contenção; contêineres sem este parâmetro explícito acabam com um valor padrão baixo e, por isso, sempre é bom definir as requisições manualmente de acordo com o conhecimento de requisições esperadas de cada serviço, assim evitando que o pod seja tratado como de baixa prioridade de CPU quando a disponibilidade do serviço precisa ser alta.

Também existem ganhos claros de escalabilidade quando a linguagem de programação adotada na aplicação tem mecanismos de tolerância a falhas e manutenção da saúde da aplicação considerando um volume elevado de requisições e limites de consumo da infraestrutura subjacente.

O CFS assegura que, no longo prazo, cada processo receba CPU proporcional à sua parcela. Contudo, ao colocar um *limite* (perceba que o termo equivale a “quota de processamento”), ativamos uma restrição rígida: o cgroup só pode consumir X microsegundos de CPU por período de 100 milissegundos. Se atingir essa disponibilidade antes do período acabar, o processo fica “congelado” até o próximo tick – daí vem o *throttling* observado. Em situações normais, se todos os processos têm uso $\leq$ requisições, o CFS lida bem; o limite só “ativa” mesmo se o serviço tentar exceder essa quota predefinida. Esse mecanismo evita *starvation* (fome) de recursos e garante **isolamento** entre contêineres – um princípio de *multitenancy* herdado de máquinas virtuais.

Assim, podemos dizer que Kubernetes acrescenta uma camada de **escalonamento justo entre pods** usando cgroups e políticas de QoS. Por exemplo,

pods com qualidade de serviços configuradas como “burstable” (estouráveis, que têm quota de requisições menores que os limites de recursos) podem usar recursos ociosos além de sua quota de requisições até o limite, mas sob contenção, onde o agendador do kernel prioriza quem está dentro da quota de requisição. Já pods *Guaranteed* (request == limit para todos os contêineres) têm ainda mais garantia de servir integralmente o que pediram, já que existe um ajuste fino entre o limite de recursos e a quantidade de requisições de processamento.

No nível do cluster, outro conceito importante é a **Preempção de Pods**. Kubernetes introduziu o recurso de *Pod Priority and Preemption* para lidar com cenários em que não há recursos livres para agendar um pod crítico. Cada pod pode ter uma *PriorityClass* (classe de prioridade) numérica; se um novo pod de alta prioridade não encontra espaço, o agendador pode **preemptar (evict)** pods de menor prioridade para abrir espaço. Ou seja, o Kubernetes “mata” alguns pods menos importantes (enviando SIGTERM e respeitando *graceful termination*, que encerra os processos liberando memória e impedindo serviços em background de manterem o funcionamento mesmo que não estejam ativos) para liberar CPU/memória para o pod prioritário ser escalonado. Isso lembra algoritmos de escalonamento com prioridade em sistemas operacionais, só que aqui a “tarefa” é um contêiner inteiro.

A preempção respeita certas regras, por exemplo: tenta não violar *PodDisruptionBudget* e prefere remover pods *BestEffort* ou *Burstable* antes de pods *Guaranteed* – de fato, o Kubelet ao lidar com pressão de memória local já segue essa lógica: ele evita primeiro pods QoS **BestEffort** (aqueles sem requisições de recursos) que são considerados menos críticos, depois *Burstable* e só em último caso sacrifica pods *Guaranteed*. Essa política evita que um pod vital (digamos, seu banco de dados) seja morto antes de um pod de teste ou sem importância, mesmo que ambos estejam consumindo muita memória.

Em essência, temos aqui aplicação do conceito de **prioridade com preempção**, clássico em escalonadores de CPU, adaptado para orquestração de contêineres no cluster. A configuração de *PriorityClasses* nos permite implementar estratégias de **QoS global**: por exemplo, marcar pods do sistema (DNS, Metrics Server) com prioridade ultra alta (as *PriorityClasses* padrão **system-cluster-critical** e **system-node-critical** têm valores enormes) para que nunca sejam preemptados ou evitados antes de outros. Enquanto isso, cargas de *processamento em lote* ou

ambientes de desenvolvimento podem ter prioridade baixa e ser preemptadas quando faltar recurso – isso habilita um uso eficiente do cluster, rodando jobs low-priority em segundo plano que cedem lugar caso cargas de trabalho mais importantes precisem.

Por fim, falemos de **garbage collection no contexto de Kubernetes**, mas agora no sentido de recursos do cluster. O Kubernetes controla o ciclo de vida de objetos e tem seu próprio “GC” para containers e imagens. O kubelet faz *garbage collect* de contêineres finalizados e imagens não utilizadas para liberar espaço em disco. Já no plano de controle, há um *Garbage Collector* de objetos do etcd que remove recursos órfãos (por exemplo, ReplicaSets antigos quando um Deployment é deletado, ou recursos cuja disponibilidade não é crítica para a requisição).

Esses mecanismos automáticos evitam vazamentos de recursos (um análogo a vazamentos de memória, mas aqui vazamento de objetos no cluster). A sintonia fina desses parâmetros (quantos pods finalizados manter, percentual mínimo de disco antes de limpar imagens etc.) fica nas configs do kubelet, mas normalmente padrões saneados já previnem problemas. O importante é entender que *garbage collection* acontece tanto dentro das apps (gerenciando memória) quanto no nível da orquestração (gerenciando contêineres e objetos), garantindo que o cluster se mantenha “limpo” e performático ao longo do tempo.

Estatísticas, algoritmos de health-check e balanceamento de carga

Monitorar e otimizar aplicações em Kubernetes envolve também *análise estatística de métricas* e aplicação de algoritmos inteligentes de roteamento de tráfego. Um conceito difundido pelo Google SRE são os **Quatro Sinais de Ouro (Golden Signals)**: latência, tráfego, erros e saturação. Eles servem como base para avaliar a saúde de sistemas complexos.

Em Kubernetes, nossas probes de liveness/readiness tratam principalmente do sinal “saturação/erro”: se o serviço está saturado a ponto de não responder ou se encontrou um erro fatal, as probes capturam isso. Mas para ter visibilidade completa, é importante coletar métricas de latência de requisições, throughput (Requisições por segundo) e taxas de erro protocolares (HTTP 5XX, por exemplo). Ferramentas como **Prometheus + Grafana** ou serviços gerenciados (Datadog, New Relic, Azure Monitor) ajudam a aplicar estatística: podemos visualizar, por exemplo, um *histograma de*

latência e notar que quando o uso de CPU passa de 80%, o percentil 95% de latência dobra – indicando correlação entre saturação de CPU e tempo de resposta.

Com esses dados, podemos definir *thresholds* para autoescalonamento horizontal (HPA): “se CPU > 70% por 5 minutos, adicione mais pod” – evitando assim chegar na saturação total que degrade a qualidade. Ou ainda usar *custom metrics*: número de requisições em fila, uso de memória, uso de CPU, quantidade de *staled processes*, entre outros, alimentando algoritmos de decisão (que podem utilizar ou não monitoramento em tempo real por Inteligência Artificial).

No contexto de **algoritmos de verificação de estado**, já mencionamos que Kubernetes adota um método simples (checagens periódicas com contagem de falhas consecutivas). Alternativas mais sofisticadas existem: por exemplo, um algoritmo chamado **Phi Accrual Failure Detector** calcula um valor φ baseado nos intervalos de heartbeat para estimar a probabilidade de um nó ter caído, em vez de usar um limiar fixo de tentativas.

Ele é usado em sistemas como Cassandra e Akka. Poderíamos imaginar um operador Kubernetes customizado implementando algo assim para decidir se um pod está saudável com base em latência ou jitter dos heartbeats, mas o ganho talvez não justifique a complexidade em muitos cenários. O Kubernetes optou por um design determinístico e fácil de entender – o que facilita para operadores ajustarem `initialDelaySeconds`, `periodSeconds` e `failureThreshold` conforme cada caso (por exemplo, aplicações CPU-bound sob carga podem precisar aumentar timeouts para evitar falsos negativos de liveness).

A comunidade OpenShift (Red Hat) recomenda bastante separar **liveness vs readiness**: liveness só deve detectar travamentos irrecuperáveis (como deadlocks) e não ficar sensível a lentidão transitória; readiness sim deve ser sensível para tirar o pod do balanceador de carga se ele não conseguir responder em tempo hábil. Esse delineamento também aparece na prática de *circuit breaker*: ao invés de derrubar o pod quando uma dependência externa falha, às vezes é melhor o app retornar erros (por exemplo, em requisições HTTP, código 503) rapidamente – assim o cliente recebe uma falha, mas a instância permanece viva e pode se recuperar quando a dependência voltar, evitando um ciclo de reinícios.

Este é um ponto de constante debate na comunidade de engenharia de software. Alguns sugerem não colocar checagem de banco de dados contínua na *readiness probe* justamente por isso – se o banco de dados cair (enquanto dependência essencial do serviço), talvez o melhor seja o app “degradar graciosamente” e responder com erros/transações pendentes, ao invés de se remover do serviço e possivelmente causar falha em cascata se **todas** as instâncias fizerem o mesmo.

Por outro lado, para *startup probe* é válido checar dependências (não subir completamente se não conectar no banco de dados, por exemplo), mas após iniciado, usar estratégias de tolerância a falhas em vez de simplesmente terminar todos os processos (graciosamente). Esse equilíbrio requer entendimento estatístico: qual a probabilidade de falha temporária do serviço de dependência? Quanto tempo em média o serviço do POD aguenta sem a dependência? Quais os mecanismos de tolerância a falha implementados para garantir a sobrevivência do serviço em estado crítico?

Com base nesse monitoramento constante com coleta de dados e avaliação contínua da qualidade dos serviços, é possível fazer ajustes finos de escalabilidade e disponibilidade, da mesma forma como incluir estratégias na aplicação, estruturas de redundância para dependência de serviços críticos, e finalmente a configuração de condições de timeouts e comportamento das probes.

No que tange a **balanceamento de carga**, Kubernetes por padrão usa um algoritmo simples de distribuição: no Service (modo ClusterIP), o kube-proxy (iptables ou IPVS) efetua algo próximo de round-robin aleatório entre as endpoints disponíveis. Ou seja, cada nova conexão ou fluxo é encaminhada a uma pod preparada para o processamento de forma equilibrada, respeitando a distribuição para otimizar a eficiência de processamento de acordo com a disponibilidade de recursos.

Isso é geralmente suficiente, mas há estratégias mais avançadas suportadas indiretamente: por exemplo, *Ingress Controllers* ou *Service Meshes* (como Istio/Envoy) podem implementar o **balanceamento de carga do lado do usuário** com algoritmos de alto nível: *least connections*, *least response time*, *EWMA latency*, entre outros.

Em ambientes de tráfego intenso, podemos querer desviar carga de pods que, embora estejam *Ready*, estejam com resposta mais lenta que seus pares – indicando

saturação próxima por uso excessivo de recursos ou configuração de *probes* e configurações de recurso limitantes. Com isso, a configuração das *probes*, requisições e limites devem ser pareados com os ajustes de tráfego para a aplicação a nível externo, reduzindo a quantidade de requisições que devem ser processadas do lado do cluster para reduzir o stress de operacionalização e gerenciamento das cargas de trabalho entre os pods.

Algoritmos de balanceamento adaptativo usam estatística contínua, medindo a latência média de cada pod e distribuindo menos tráfego para os *outliers*, exigindo uma proxy do lado do cliente que implemente um mecanismo de distribuição de carga nas camadas de transporte e aplicação, minimizando o *overhead* de orquestração do cluster.

Algoritmos de *balanceamento adaptativo* são estratégias de alocação dinâmica de requisições em múltiplos servidores (ou pods), onde a escolha do destino não é fixa nem estática, mas depende de variáveis estocásticas observadas em tempo real, como carga, latência ou *throughput*. Formalmente, considere n servidores $S_1, \dots, S_n$ e uma sequência de requisições R_t ($t = 1, 2, \dots$). Um algoritmo adaptativo define uma distribuição de probabilidade condicional:

$$P(R_t \rightarrow S_i | X_{t-1})$$

Onde X_{t-1} é o vetor de métricas observadas até o instante $t - 1$ (uso de CPU, filas, tempos médios de resposta). Diferente do balanceamento estático (como *Round Robin*), em que $P(R_t \rightarrow S_i) = 1/n$, aqui as probabilidades são atualizadas iterativamente a partir de estimadores $\hat{\theta}_i(t)$ das condições de cada servidor.

Esse processo é estocástico, dependente do histórico e converge, sob hipóteses de ergodicidade, para alocações que minimizam tempos médios de resposta ou maximizam *throughput*. Em termos de otimização online, trata-se de um problema de *multi-armed bandits* adaptativo, onde cada servidor é um “braço” e os pesos são ajustados segundo recompensas observadas (baixa latência, alta confiabilidade).

Outra técnica estatística é o **outlier detection** (detecção de outlier): o Envoy, por exemplo, pode automaticamente marcar um pod como temporariamente *eject* do pool se suas últimas N requisições tiveram latência muito acima da média dos demais – atuando quase como uma readiness probe dinâmica baseada em percentis de

latência. Tudo isso são aplicações práticas de estatística para melhorar disponibilidade percebida.

Podemos citar também **algoritmos de escalonamento e controle**: o Horizontal Pod Autoscaler (HPA) do Kubernetes usa um controlador que implementa essencialmente um laço de controle *feedback*: ele monitora a métrica alvo (por padrão, CPU utilizada média) e ajusta o número de réplicas para mantê-la no *setpoint* desejado. Internamente, o HPA calcula a média atual e compara com o target, decidindo escalar para cima ou para baixo proporcionalmente. Há um pequeno atraso e um histórico (o HPA espera alguns minutos antes de escalar para baixo, para evitar *flapping*). São conceitos de teoria de controle básicos (um controlador PI simplificado) aplicados a sistemas distribuídos, onde a “planta” é a carga de trabalho e o “atuador” é o número de pods.

Um ponto divertido de mencionar: existe até *controle preditivo* sendo estudado para Kubernetes – usar algoritmos de aprendizado de máquina para prever aumento de carga e escalar antes que a latência suba. Alguns trabalhos de pesquisa exploram usar séries temporais e processos markovianos para prever estado futuro do cluster e antecipar ações (como *proactive scaling*). Embora avançados, esses ainda não são padrão nas ferramentas, mas quem sabe no futuro.

MERCADO, CASES E TENDÊNCIAS

Para fechar nosso material, vamos analisar exemplos reais de empresas que obtiveram sucesso (ou sofreram lições difíceis) ao implementar probes de saúde e gerenciamento de recursos em Kubernetes. Histórias do mundo real ajudam a cristalizar a importância desses conceitos e fornecer insights práticos. Além disso, indicaremos algumas referências – livros, artigos e vídeos – onde você pode se aprofundar nesses casos.

Shopify e Readiness Probes (sucesso na migração de Kafka)

A Shopify, conhecida plataforma de e-commerce, compartilhou em seu Engineering Blog a experiência de rodar Apache Kafka no Kubernetes. Um dos desafios era realizar implantações e atualizações de configuração sem interrupção em clusters Kafka que processam bilhões de mensagens. A solução envolveu utilizar **StatefulSets** com pods Kafka e uma inteligência com **readiness probes**: cada broker Kafka só se tornava *Ready* (do ponto de vista do Service) após concluir sua sincronização de dados e estar pronto para reassumir carga.

Durante um rolling update, o Kubernetes atualizava um pod por vez; a readiness probe garantia que, após um broker reiniciar com nova config, ele não recebesse tráfego de produtores/consumidores até que estivesse totalmente operacional no cluster (p. ex., já reintegrado ao quorum e sem lag de réplicas). Esse uso coordenado de readiness permitiu “controlar a velocidade” do rollout – essencialmente, o cluster Kafka ficava sempre em estado saudável, pois nunca havia mais de um broker indisponível por vez, e mesmo este não recebia clientes até estar pronto.

O resultado foi uma migração para nuvem bem-sucedida, sem downtime perceptível. A Shopify demonstrou assim o poder das readiness probes combinadas com a semântica do **StatefulSet** (ordem e espera entre pods): aumentou a confiabilidade da plataforma de mensageria. Esse caso foi apresentado em detalhes num artigo *“Running Kafka on Kubernetes at Shopify”* e é frequentemente citado como referência de arquitetura resiliente. Para quem se interessar, recomendo assistir ao talk do Shopify na KubeCon 2018 sobre esse assunto e ler o case study no blog da empresa.

GitLab e Limits de Memória (lições aprendidas)

O GitLab, plataforma DevOps, inicialmente enfrentou problemas ao executar sua pesada suíte de serviços em Kubernetes. Um caso emblemático foi com o **Sidekiq**, componente de processamento assíncrono. Em certo momento, o GitLab observou *OOMKills* frequentes nos pods do Sidekiq – eles haviam configurado um limite de memória muito baixo, herança de um “memory killer” interno da aplicação (que finalizava workers acima de ~2GB).

No ambiente Kubernetes (sem swap e com OOM Killer do kernel ativo), isso levou a reinicializações abruptas de pods em produção quando a memória passava do limite rígido, interrompendo jobs no meio. A equipe do GitLab reagiu aumentando significativamente os **memory limits** para valores entre 3 e 8 GB por pod, acomodando picos de uso do Ruby sem deixar o kernel matar o processo. Além disso, eles melhoraram o gerenciamento dentro da app: implementaram o **Sidekiq MemoryKiller**, um watchdog em Ruby que monitora o RSS de cada worker e, se exceder um certo threshold por algum tempo, faz um shutdown gracioso daquele processo, liberando memória antes que o kernel o mate.

Essa estratégia dupla – limits de memória realistas no Kubernetes + autocontrole de memória na aplicação – estabilizou o consumo e reduziu drasticamente incidências de OOM. É um exemplo de falha inicial por má configuração (limits mal ajustados) que foi corrigida combinando ajuste de recursos do Kubernetes com engenharia na aplicação. Em posts de retrospectiva, times do GitLab também enfatizaram a importância de monitorar continuamente o uso real vs. requests/limits e adequar valores. Eles passaram a usar ferramentas para autosuggest de requests com base no histórico (ex.: Kube-native metrics).

Um engenheiro do GitLab publicou um artigo no [Medium](#) intitulado “Resource Management in GitLab’s CI on Kubernetes” detalhando essas práticas e recomendando definir **reservas (requests) para absolutamente todos os contêineres** – pois, no início, alguns pods rodavam sem requests e eram classificados como *BestEffort*, ficando vulneráveis a serem expurgados primeiro em cenários de pressão de memória. A lição do GitLab: **não subestime** o consumo de memória de componentes complexos, e use as ferramentas de Kubernetes (Limits, LimitRanges, monitoramento) para evitar tanto crashes quanto desperdício. Hoje o GitLab

disponibiliza inclusive recomendações oficiais de sizing para quem instala on-premises, enfatizando memória adequada e configuração do Memory Killer interno.

Buffer e CPU Throttling (repensando limites)

Já mencionamos a história da Buffer, mas vale reforçar como estudo de caso. A Buffer é uma plataforma de gerenciamento de mídias sociais que migrou para microserviços em Kubernetes. Seguindo boas práticas comuns, eles definiram limites de CPU em todos os seus deployments para evitar cenários de sobrecarga. Entretanto, eles notaram algo curioso: alguns serviços apresentavam latências elevadas, embora os gráficos mostrassem baixo uso de CPU. Investigando com métricas detalhadas do kernel, descobriram que esses pods estavam sofrendo **throttling de CPU milhares de vezes por segundo**, mesmo usando apenas ~20% da CPU concedida!

A causa era um bug conhecido no algoritmo de CFS quota em certas versões do kernel, que faz com que mesmo cargas espinhosas de curto prazo possam acionar o throttling antes da hora. Em termos práticos, o serviço não ultrapassava 0.2 cores de uso médio, mas pequenos *bursts* eram o suficiente para consumir rapidamente a cota de 100ms e depois o processo ficava aguardando o próximo período – resultando em latência acrescida.

A solução ousada da Buffer foi **remover os CPU limits** de (quase) todos os serviços de produção, mantendo apenas os requests para garantir distribuição de carga nos nós. Claro que isso traz o risco de um pod eventualmente usar mais CPU do que deveria e impactar co-localizados; para mitigar, eles isolaram pods sem limit em nós dedicados via *taints/tolerations* (nós “unbounded”). Também aumentaram os requests desses pods para refletir picos observados, garantindo scheduling correto. Após essas mudanças, mediram ganhos impressionantes: vários endpoints tiveram redução de p99 latência em 5-10x, e o caso extremo foi a página principal (landing page) que ficou 22x mais rápida.

Este caso ficou famoso a ponto de ser citado num tópico do Hacker News e em uma palestra do Airbnb sobre falhas em Kubernetes. A Buffer basicamente desafiou a recomendação padrão de “sempre colocar limits” – mostrando que, em determinados casos, **menos é mais**. A contrapartida é que você precisa de um nível alto de observabilidade para garantir que um serviço sem limit não degrade todo o

cluster. A experiência deles foi documentada no blog post “*Make your services faster by removing CPU limits*” (Eric Khun, 2020).

Depois dessa publicação, a comunidade debateu e o próprio Kubernetes.io publicou um artigo respondendo, intitulado “*The Case for CPU Limits: Predictability vs Efficiency*”, reconhecendo o ponto da Buffer mas salientando que em clusters multi-tenant não controlados, limites ainda são importantes para isolamento. Em suma, esse estudo de caso ilumina a decisão de tuning: se a prioridade é **desempenho de baixa latência** e você confia no isolamento por outras vias, pode relaxar limites; se a prioridade é **previsibilidade e fairness** entre muitos serviços, mantenha-os – ou busque um meio-termo (por ex., limites bem acima do request, permitindo bursts moderados).

O QUE VOCÊ VIU NESTA AULA?

Esta primeira aula do curso **Kubernetes Avançado** explorou a fundo como garantir a saúde de aplicações e gerenciar recursos no cluster. Vimos que liveness/readiness/startup probes são ferramentas poderosas para atingir **autocura** e **alta disponibilidade**, enquanto requests/limits e afinamentos de CPU/Memória garantem **isolamento** e **desempenho estável** no ambiente compartilhado.

Combinamos teoria e prática, do Rust ao Markov, do YAML ao GC, passando por casos reais do mercado – tudo para equipar você, arquiteto ou engenheiro, a desenhar sistemas cada vez mais resilientes e eficientes sobre o Kubernetes. Nas próximas aulas, continuaremos nessa pegada de mesclar hands-on com reflexão avançada, cobrindo outros tópicos de Kubernetes Avançado.

Por ora, ficam as lições: monitore tudo, teste suas probes exaustivamente e lembre-se que cada parâmetro escondido no manifest pode ser a diferença entre uma noite de sono tranquila ou um incidente de madrugada! Boas práticas e estudo contínuo são a cura para a maioria dos “males” de clusters – e agora você já conhece os remédios iniciais para manter suas aplicações saudáveis e seus recursos bem gerenciados. **Bons deploys!**

REFERÊNCIAS

Ewaschuk, Rob. “**Monitoring Distributed Systems.**” In *Site Reliability Engineering*. Google. Disponível em: <https://sre.google/sre-book/monitoring-distributed-systems/>. Acesso em: 04 set. 2025.

GitLab. “**Reducing Memory Use.**” *GitLab Docs*. Disponível em: https://docs.gitlab.com/administration/sidekiq/sidekiq_memory_killer/. Acesso em: 04 set. 2025.

GitLab. “**Sidekiq Memory Limits and Memory Killer in Kubernetes (#1084).**” *GitLab Infrastructure Team – Issues*. Disponível em: <https://gitlab.com/gitlab-com/gl-infra/delivery/-/issues/1084>. Acesso em: 04 set. 2025.

Hun, Erick. “**Kubernetes: Make Your Services Faster by Removing CPU Limits.**” Disponível em: <https://erickhun.com/posts/kubernetes-faster-services-no-cpu-limits/>. Acesso em: 04 set. 2025.

Kubernetes. “**Assign Memory Resources to Containers and Pods.**” *Kubernetes Documentation*. Disponível em: <https://kubernetes.io/docs/tasks/configure-pod-container/assign-memory-resource/>. Acesso em 04 set. 2025.

Kubernetes. “**Secrets.**” *Kubernetes Documentation*. Disponível em: <https://kubernetes.io/docs/concepts/configuration/secret/>. Acesso em 27 set. 2025.

Lukša, Marko. *Kubernetes in Action*. Shelter Island, NY: Manning Publications, 2018. ([Manning Publications](#), [AbeBooks](#), [Google Livros](#))

PALAVRAS-CHAVE

Kubernetes. Probes. Management.

EMANIP



POSTECH